Universidad Andrés Bello Online

Guía de Ejercicios
Unidad 1

Guía de Ejercicios Unidad 1

Guía de Ejercicios Unidad

Ejercicio 1: Gestión de un inventario de productos (listas y diccionarios)
Desarrollo Sugerido:

1. Crea una lista llamada inventario que contenga tres diccionarios. Cada diccionario debe representar un producto con las siguientes claves: "nombre", "precio" y "stock". Asigna valores de tu elección.

2. Añade un cuarto producto a la lista inventario usando el método append().

3. Modifica el *stock* del segundo producto en la lista inventario para que sea 15.

4. Calcula el valor total del inventario (precio * *stock*) para cada producto y guarda los resultados en una nueva lista llamada valor_total.

5. Imprime la lista inventario final y la lista valor_total.

Solución Esperada: esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 1

Ejercicio 1 Unidad 1.

```python
# 1. Crea una lista llamada inventario que contenga tres diccionarios.
inventario = [
    {"nombre": "Laptop", "precio": 1200, "stock": 10},
    {"nombre": "Teclado", "precio": 75, "stock": 20},
    {"nombre": "Mouse", "precio": 25, "stock": 30}
]

# 2. Añade un cuarto producto a la lista inventario usando el método append().
nuevo_producto = {"nombre": "Monitor", "precio": 300, "stock": 15}
inventario.append(nuevo_producto)

# 3. Modifica el stock del segundo producto en la lista inventario para que sea 15.
inventario[1]["stock"] = 15

# 4. Calcula el valor total del inventario (precio * stock) para cada producto
# y guarda los resultados en una nueva lista llamada valor_total.
valor_total = []
for producto in inventario:
    valor_total_producto = producto["precio"] * producto["stock"]
    valor_total.append(valor_total_producto)

# 5. Imprime la lista inventario final y la lista valor_total.
print("Inventario final:")
print(inventario)
print("\nValor total por producto:")
print(valor_total)
```

Ejercicio 2: Registro de tareas pendientes (archivos de texto y with)
Desarrollo Sugerido:

1. Crea un programa que pida al usuario que ingrese tres tareas pendientes. Cada tarea debe ser una línea de texto.
2. Usa la sentencia with open(...) en modo de escritura ("w") para crear un archivo llamado "tareas.txt". Escribe cada una de las tres tareas en una línea separada.

3. Abre el mismo archivo, pero esta vez en modo de lectura ("r") con la sentencia with open(...).
4. Lee el contenido del archivo línea por línea y muestra cada tarea en la consola, asegurándote de eliminar los saltos de línea adicionales.
5. Añade un bloque try...except para manejar la posibilidad de que el archivo no exista al intentar leerlo.

Solución Esperada: esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 2

Ejercicio 2 Unidad 1

```python
# 1. Crea un programa que pida al usuario que ingrese tres tareas pendientes.
tareas = []
for i in range(3):
    tarea = input(f"Ingresa la tarea {i+1}: ")
    tareas.append(tarea)

# 2. Usa la sentencia with open(...) en modo de escritura ("w") para
# crear un archivo llamado "tareas.txt". Escribe cada una de las tres
# tareas en una línea separada.
with open("tareas.txt", "w") as f:
    for tarea in tareas:
        f.write(tarea + "\n")

# 3. Abre el mismo archivo, pero esta vez en modo de lectura ("r")
# con la sentencia with open(...).
# 5. Añade un bloque try...except para manejar la posibilidad de que el archivo
# no exista al intentar leerlo.
try:
    with open("tareas.txt", "r") as f:
        print("\nTareas pendientes:")
    # 4. Lee el contenido del archivo línea por línea y muestra cada tarea
    # en la consola, asegurándote de eliminar los saltos de línea adicionales.
        for linea in f:
            print(linea.strip())
except FileNotFoundError:
    print("El archivo 'tareas.txt' no fue encontrado.")
```

Ejercicio 3: Análisis de Datos CSV (Módulos y try...except)
Desarrollo Sugerido:

1. Crea manualmente un archivo de texto llamado "ventas.csv" con el siguiente contenido:

producto,cantidad,precio
manzana,50,1.5
banana,100,0.75
naranja,75,1.2

2. Crea un programa de Python que lea el archivo "ventas.csv" usando el módulo csv.
3. Calcula el ingreso total por cada producto (cantidad * precio) y almacena los resultados en un diccionario donde la clave sea el nombre del producto y el valor sea el ingreso.
4. Añade un bloque try...except para manejar la posibilidad de que el archivo "ventas.csv" no se encuentre (FileNotFoundError).
5. Muestra el diccionario de ingresos por producto en la consola

Solución Esperada: esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 3

Ejercicio 3 Unidad 1.

```python
import csv

# 1. Crea manualmente un archivo de texto llamado
# "ventas.csv" con el siguiente contenido:
# ... Asumiendo que el archivo ya ha sido creado

# 2. Crea un programa de Python que lea el archivo "ventas.csv"
# usando el módulo csv.
# 3. Calcula el ingreso total por cada producto (cantidad * precio)
# y almacena los resultados en un diccionario.
# 4. Añade un bloque try...except para manejar la posibilidad de que
# el archivo "ventas.csv" no se encuentre (FileNotFoundError).
# 5. Muestra el diccionario de ingresos por producto en la consola

ingresos_por_producto = {}

try:
    with open("ventas.csv", mode='r', newline='') as archivo_csv:
        lector_csv = csv.reader(archivo_csv)
        next(lector_csv)  # Omitir la fila de encabezado

        for fila in lector_csv:
            producto = fila[0]
            cantidad = int(fila[1])
            precio = float(fila[2])
            ingreso = cantidad * precio
            ingresos_por_producto[producto] = ingreso

    print("Ingresos totales por producto:")
    print(ingresos_por_producto)

except FileNotFoundError:
    print("Error: El archivo 'ventas.csv' no fue encontrado.")
except ValueError:
    print("Error: Hubo un problema al convertir datos a número.")
    print("Asegúrate de que cantidad y precio contengan solo valores numéricos")
except Exception as e:
    print(f"Ocurrió un error inesperado: {e}")
```

Universidad Andrés Bello

Ejercicio 4: Manejo de opciones de configuración json (json y from...import)
Desarrollo Sugerido:

1. Crea un programa que defina un diccionario de configuración. Por ejemplo:

```
configuracion = {
"tema": "oscuro",
"idioma": "es",
"notificaciones": True,
"usuarios_online": ["user1", "user2"]
}
```

2. Usa la sentencia with y la función json.dump() para guardar este diccionario en un archivo llamado "config.json". Asegúrate de usar el parámetro indent=4 para que el archivo sea legible.
3. Crea un segundo programa que use with y json.load() para leer el archivo "config.json".
4. Imprime el valor de la clave "idioma" y el primer elemento de la lista "usuarios_online".
5. Añade un try...except para manejar un posible JSONDecodeError si el archivo no es válido.

Solución Esperada: esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 4

Ejercicio 4 Unidad 1.

```python
import json

# 1. Crea un programa que defina un diccionario de configuración.
configuracion = {
    "tema": "oscuro",
    "idioma": "es",
    "notificaciones": True,
    "usuarios_online": ["user1", "user2"]
}

# 2. Usa la sentencia with y la función json.dump() para guardar este
# diccionario en un archivo llamado "config.json".
# Asegúrate de usar el parámetro indent=4 para que el archivo sea legible.
try:
    with open("config.json", "w") as archivo_json:
        json.dump(configuracion, archivo_json, indent=4)
    print("Diccionario de configuración guardado en 'config.json'")

    # 3. Crea un segundo programa que use with y json.load() para leer el
    # archivo "config.json".
    # 5. Añade un try...except para manejar un posible JSONDecodeError si
    # el archivo no es válido.
    with open("config.json", "r") as archivo_json:
        configuracion_leida = json.load(archivo_json)
    print("\nDiccionario de configuración leído desde 'config.json':")
    print(configuracion_leida)

    # 4. Imprime el valor de la clave "idioma" y el primer elemento de la lista
    # "usuarios_online".
    print(f"\nValor de 'idioma': {configuracion_leida.get('idioma', 'Clave no encontrada')}")
    usuarios_online = configuracion_leida.get('usuarios_online', [])
    if usuarios_online:
        print(f"Primer elemento de 'usuarios_online': {usuarios_online[0]}")
    else:
        print("La lista 'usuarios_online' está vacía o la clave no existe.")

except FileNotFoundError:
    print("Error: El archivo 'config.json' no fue encontrado.")
except json.JSONDecodeError:
    print("Error: El archivo 'config.json' no es un archivo JSON válido.")
except Exception as e:
    print(f"Ocurrió un error inesperado: {e}")
```

Ejercicio 5: Generador de contraseñas (módulos random y string)

Desarrollo Sugerido:

1. Crea un programa que importe los módulos random y string.
2. Define una variable caracteres que sea una combinación de letras minúsculas, mayúsculas, dígitos y signos de puntuación usando las constantes del módulo string. *Pista: puedes usar string.ascii_letters, string.digits y string.punctuation y concatenarlas.*
3. Pide al usuario que ingrese la longitud deseada de la contraseña (por ejemplo, 12).
4. Usa un bucle y random.choice() para generar una contraseña de la longitud especificada, seleccionando caracteres al azar de la variable caracteres.
5. Imprime la contraseña generada.

Solución Esperada: esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 5

Ejercicio 5 Unidad 1.

```python
import random
import string

# Define una variable caracteres que sea una combinación de letras minúsculas,
# mayúsculas, dígitos y signos de puntuación usando las constantes del módulo string.
caracteres = string.ascii_letters + string.digits + string.punctuation

# Pide al usuario que ingrese la longitud deseada de la contraseña
try:
    longitud = int(input("Ingresa la longitud deseada de la contraseña: "))

    # Usa un bucle y random.choice() para generar una contraseña de la longitud especificada
    contrasena = ""
    for _ in range(longitud):
        contrasena += random.choice(caracteres)

    # Imprime la contraseña generada.
    print(f"Contraseña generada: {contrasena}")

except ValueError:
    print("Entrada inválida. Por favor, ingresa un número entero para la longitud.")
except Exception as e:
    print(f"Ocurrió un error: {e}")
```